



Carlos Cotrini

Building ML AI Applications

Course from D-INFK at ETH Zurich

Spring 2026

1. Reinforcement Learning

From one delivery van to the Bellman equation: decisions under uncertainty that pay off over months.

One van, one call per week

- You run a delivery business with a single van: **Old Bessie**.

One van, one call per week

- You run a delivery business with a single van: **Old Bessie**.
- Each week she sits at one of four wear levels, and you make **exactly one call**:
 - **run** the route: collect the week's profit, risk a breakdown, add wear
 - **service** her: pay for a week in the shop, she fails less often afterwards
 - **replace** her: a large expense that buys a fresh start

One van, one call per week

- You run a delivery business with a single van: **Old Bessie**.
- Each week she sits at one of four wear levels, and you make **exactly one call**:
 - **run** the route: collect the week's profit, risk a breakdown, add wear
 - **service** her: pay for a week in the shop, she fails less often afterwards
 - **replace** her: a large expense that buys a fresh start
- Every option trades a **short-term cost** against a **long-term payoff**, and chance interferes at every step.

Decisions that pay off over months

- What is decided today changes what must be decided tomorrow.
- Outcomes are uncertain: the same decision may turn out well one week and badly the next.

Decisions that pay off over months

- What is decided today changes what must be decided tomorrow.
- Outcomes are uncertain: the same decision may turn out well one week and badly the next.
- **Reinforcement learning**: a framework for computing decision strategies that maximize the reward accumulated over time in precisely such problems.

Decisions that pay off over months

- What is decided today changes what must be decided tomorrow.
- Outcomes are uncertain: the same decision may turn out well one week and badly the next.
- **Reinforcement learning**: a framework for computing decision strategies that maximize the reward accumulated over time in precisely such problems.
- Our single running example today: the **repair-or-replace** problem of the interactive visualization at

`sml-fs26.github.io/classic_rl/repair-or-replace`

➡ Live demo: the title scene and “You run the fleet”

The model: Markov decision processes

Definition: Markov decision process (MDP)

A package of four ingredients (S, A, T, γ) :

- S : the set of **states** the system can be in
- A : the set of **actions** available to the decision maker
- T : the **transition dynamics**, describing how the system reacts when an action is taken
- γ : the **discount factor**, a number between 0 and 1

The model: Markov decision processes

Definition: Markov decision process (MDP)

A package of four ingredients (S, A, T, γ) :

- S : the set of **states** the system can be in
 - A : the set of **actions** available to the decision maker
 - T : the **transition dynamics**, describing how the system reacts when an action is taken
 - γ : the **discount factor**, a number between 0 and 1
- S and A are defined **a priori**: fixed in advance, part of the problem description. Writing them down is the modeler's first task. T is then defined on top of them.
- γ 's use will be explained later.

The transition dynamics is a randomized function

- T consumes a state s and an action a , and produces a **new state** s' and a **reward** r :

$$(s', r) = T(s, a).$$

- In words: feed in where you are and what you do; out come where you land and the immediate gain or loss of that step.

The transition dynamics is a randomized function

- T consumes a state s and an action a , and produces a **new state** s' and a **reward** r :

$$(s', r) = T(s, a).$$

- In words: feed in where you are and what you do; out come where you land and the immediate gain or loss of that step.

The dynamics are random

Feeding the *same* state and action (s, a) in twice may produce *different* new states and different rewards.

The transition dynamics is a randomized function

- T consumes a state s and an action a , and produces a **new state** s' and a **reward** r :

$$(s', r) = T(s, a).$$

- In words: feed in where you are and what you do; out come where you land and the immediate gain or loss of that step.

The dynamics are random

Feeding the *same* state and action (s, a) in twice may produce *different* new states and different rewards.

- Fleet: feeding (WORN, RUN) into T may return (SHAKY, +72) one week and (FAILING, -208) another.

Example: the repair-or-replace MDP

$$S = \{\text{HEALTHY}, \text{WORN}, \text{SHAKY}, \text{FAILING}\}, \quad A = \{\text{RUN}, \text{SERVICE}, \text{REPLACE}\}$$

state	profit	breakdown
HEALTHY	+95	2 %
WORN	+72	8 %
SHAKY	+40	28 %
FAILING	+16	55 %

- RUN: earn the profit; a breakdown costs 280 on top and dumps the van to FAILING; wear slips over the weeks
- SERVICE: -50, wear often improves
- REPLACE: -130, new HEALTHY van next week
- no final week; the visualization fixes $\gamma = 0.9$

➡ Live demo: “How it works”: the fleet sheet with all the odds

Policies: your maintenance playbook

Definition: policy

A **policy** defines how to act at each state. Formally, a function $\pi: S \rightarrow A$ that takes a state s and returns the action $\pi(s)$ to be played whenever the system is in s .

Policies: your maintenance playbook

Definition: policy

A **policy** defines how to act at each state. Formally, a function $\pi: S \rightarrow A$ that takes a state s and returns the action $\pi(s)$ to be played whenever the system is in s .

Example: a playbook for the fleet

$\pi(\text{HEALTHY}) = \text{RUN}$, $\pi(\text{WORN}) = \text{SERVICE}$, $\pi(\text{SHAKY}) = \pi(\text{FAILING}) = \text{REPLACE}$

One call per wear level: a complete standing instruction for every Monday.

➡ Live demo: “Policy: your maintenance playbook”

Executing a policy: the trajectory

Definition: trajectory

The **trajectory** of a policy π is the sequence

$$s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_i, a_i, r_i, \dots$$

produced by executing π on the MDP starting from state s_1 .

Executing a policy: the trajectory

Definition: trajectory

The **trajectory** of a policy π is the sequence

$$s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_i, a_i, r_i, \dots$$

produced by executing π on the MDP starting from state s_1 .

- $a_1 = \pi(s_1)$: the first action results from applying the policy to the first state

Executing a policy: the trajectory

Definition: trajectory

The **trajectory** of a policy π is the sequence

$$s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_i, a_i, r_i, \dots$$

produced by executing π on the MDP starting from state s_1 .

- $a_1 = \pi(s_1)$: the first action results from applying the policy to the first state
- $(s_2, r_1) = T(s_1, a_1)$: the first reward and the second state result from the dynamics

Executing a policy: the trajectory

Definition: trajectory

The **trajectory** of a policy π is the sequence

$$s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_i, a_i, r_i, \dots$$

produced by executing π on the MDP starting from state s_1 .

- $a_1 = \pi(s_1)$: the first action results from applying the policy to the first state
 - $(s_2, r_1) = T(s_1, a_1)$: the first reward and the second state result from the dynamics
 - then $a_2 = \pi(s_2)$, the dynamics produce $(s_3, r_2) = T(s_2, a_2)$, and so on.
- ➡ Live demo: “The trajectory”: the six-week tape

Four weeks of Old Bessie

One execution of the playbook from $s_1 = \text{HEALTHY}$:

$s_1 = \text{HEALTHY}$	$a_1 = \text{RUN}$	$r_1 = +95$
$s_2 = \text{WORN}$	$a_2 = \text{SERVICE}$	$r_2 = -50$
$s_3 = \text{HEALTHY}$	$a_3 = \text{RUN}$	$r_3 = +95$
$s_4 = \text{HEALTHY}$	$a_4 = \text{RUN}$	$r_4 = +95$

then $s_5 = \text{WORN}$, and the walk continues, one week per step, forever.

Four weeks of Old Bessie

One execution of the playbook from $s_1 = \text{HEALTHY}$:

$s_1 = \text{HEALTHY}$	$a_1 = \text{RUN}$	$r_1 = +95$
$s_2 = \text{WORN}$	$a_2 = \text{SERVICE}$	$r_2 = -50$
$s_3 = \text{HEALTHY}$	$a_3 = \text{RUN}$	$r_3 = +95$
$s_4 = \text{HEALTHY}$	$a_4 = \text{RUN}$	$r_4 = +95$

then $s_5 = \text{WORN}$, and the walk continues, one week per step, forever.

The trajectory is random

Starting again from the same s_1 may produce a *different* trajectory: another execution might open with a first-week breakdown ($r_1 = -185$, $s_2 = \text{FAILING}$).

Scoring a trajectory: the cumulative reward

A policy is good if its trajectories accumulate as much reward as possible.

Definition: cumulative reward (the return)

Take one particular trajectory $s_1, a_1, r_1, s_2, a_2, r_2, \dots$. Its **cumulative reward** is

$$G_1 = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots + \gamma^{i-1} r_i + \dots$$

Scoring a trajectory: the cumulative reward

A policy is good if its trajectories accumulate as much reward as possible.

Definition: cumulative reward (the return)

Take one particular trajectory $s_1, a_1, r_1, s_2, a_2, r_2, \dots$. Its **cumulative reward** is

$$G_1 = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots + \gamma^{i-1} r_i + \dots$$

- In words: add up the rewards of all weeks, forever, scaling week i by γ^{i-1} . Week one counts in full, week two is scaled by γ , week three by γ^2 , and so on.
- G stands for *gain*.

Here γ plays its very important role

γ specifies **how much we care about future rewards**:

- γ close to 0: the scaling factors die out almost immediately, only near-term rewards matter
- γ close to 1: they fade slowly, rewards far in the future count almost as much as immediate ones

Here γ plays its very important role

γ specifies **how much we care about future rewards**:

- γ close to 0: the scaling factors die out almost immediately, only near-term rewards matter
- γ close to 1: they fade slowly, rewards far in the future count almost as much as immediate ones

Managers will recognize the device

It is the discounting of future cash flows from **net-present-value** calculations; as with the NPV of a perpetuity, the shrinking factors keep the never-ending total finite.

➡ Live demo: “Return over the van’s life”: drag the γ slider

The return of our four-week tape

For the trajectory we saw, with $\gamma = 0.9$:

$$G_1 = 95 + 0.9 \cdot (-50) + 0.9^2 \cdot 95 + 0.9^3 \cdot 95 + \dots$$

The return of our four-week tape

For the trajectory we saw, with $\gamma = 0.9$:

$$\begin{aligned} G_1 &= 95 + 0.9 \cdot (-50) + 0.9^2 \cdot 95 + 0.9^3 \cdot 95 + \dots \\ &= 95 - 45 + 76.95 + 69.255 + \dots \end{aligned}$$

The return of our four-week tape

For the trajectory we saw, with $\gamma = 0.9$:

$$\begin{aligned} G_1 &= 95 + 0.9 \cdot (-50) + 0.9^2 \cdot 95 + 0.9^3 \cdot 95 + \dots \\ &= 95 - 45 + 76.95 + 69.255 + \dots \end{aligned}$$

- The 50 spent on the service is felt at 90 % of its face value; the profits it enabled in weeks 3 and 4 still count for a lot.
- Exactly the arithmetic of discounted cash flows.

Start the accounting at any week

For an arbitrary week i , define G_i as the cumulative reward from week i on:

$$G_i = r_i + \gamma r_{i+1} + \gamma^2 r_{i+2} + \dots$$

Start the accounting at any week

For an arbitrary week i , define G_i as the cumulative reward from week i on:

$$G_i = r_i + \gamma r_{i+1} + \gamma^2 r_{i+2} + \dots$$

Comparing the accounting at weeks i and $i + 1$ yields a recurrence that carries the rest of this lecture:

The recurrence

$$G_i = r_i + \gamma G_{i+1}$$

In words: what is accumulated from week i on equals this week's reward plus γ times what is accumulated from week $i + 1$ on.

G_i is random, so we average it

G_i is random

Because of the randomness of the transition dynamics, G_i comes out differently after every execution starting from state s_i .

A quantity that changes from run to run cannot be maximized directly.

G_i is random, so we average it

G_i is random

Because of the randomness of the transition dynamics, G_i comes out differently after every execution starting from state s_i .

A quantity that changes from run to run cannot be maximized directly.

Definition: expectation

$\mathbb{E}[G_i]$, the **expected value** of G_i , is the **average** of G_i over a sufficiently large set of trajectories: execute the policy many times, record G_i , average the records.

G_i is random, so we average it

G_i is random

Because of the randomness of the transition dynamics, G_i comes out differently after every execution starting from state s_i .

A quantity that changes from run to run cannot be maximized directly.

Definition: expectation

$\mathbb{E}[G_i]$, the **expected value** of G_i , is the **average** of G_i over a sufficiently large set of trajectories: execute the policy many times, record G_i , average the records.

- **The ultimate objective of RL:** find the policy π maximizing $\mathbb{E}[G_1]$. Note: $\mathbb{E}[G_i]$ is a single real number, *not* random: the averaging removed the randomness.

Q^* : scoring every state-action pair

How can the best policy be discovered? Through an auxiliary function that scores every pair of a state and an action.

Definition: the Q -function

$Q^*(s, a)$ is the cumulative reward one can expect when executing an **optimal** policy from state s , where the *first* action taken there is a . In notation,

$$Q^*(s, a) = \mathbb{E}[G_i \mid s_i = s, a_i = a].$$

Q^* : scoring every state-action pair

How can the best policy be discovered? Through an auxiliary function that scores every pair of a state and an action.

Definition: the Q -function

$Q^*(s, a)$ is the cumulative reward one can expect when executing an **optimal** policy from state s , where the *first* action taken there is a . In notation,

$$Q^*(s, a) = \mathbb{E}[G_i \mid s_i = s, a_i = a].$$

- The vertical bar is read “**given that**”: the average of G_i over many executions, counting only those that are in state s at week i and take action a there, acting optimally afterwards.
- The star marks optimality.

Q^* is a table, not one number

- $\mathbb{E}[G_i]$ averages over *all* executions of the policy, whatever state they happen to be in at week i : one single number.

Q^* is a table, not one number

- $\mathbb{E}[G_i]$ averages over *all* executions of the policy, whatever state they happen to be in at week i : one single number.
- $Q^*(s, a)$ restricts the average to executions that pass through s and play a there: **one number per state-action pair**.

Q^* is a table, not one number

- $\mathbb{E}[G_i]$ averages over *all* executions of the policy, whatever state they happen to be in at week i : one single number.
- $Q^*(s, a)$ restricts the average to executions that pass through s and play a there: **one number per state-action pair**.
- For the fleet: $4 \times 3 = 12$ entries. A scorecard.
- Which week is i ? It does not matter: the fleet has no final week, so it looks the same from week 3 as from week 30.

The scorecard for the fleet

Converged values at $\gamma = 0.9$ (computed by the visualization):

state	RUN	SERVICE	REPLACE
HEALTHY	311.0	229.9	149.9
WORN	203.4	226.1	149.9
SHAKY	96.5	126.4	149.9
FAILING	-3.1	86.9	149.9

The scorecard for the fleet

Converged values at $\gamma = 0.9$ (computed by the visualization):

state	RUN	SERVICE	REPLACE
HEALTHY	311.0	229.9	149.9
WORN	203.4	226.1	149.9
SHAKY	96.5	126.4	149.9
FAILING	-3.1	86.9	149.9

- SHAKY: REPLACE beats SERVICE by 23.5. **The optimal playbook scraps a van that still starts every morning.**
- FAILING: running a dead van *loses* money on average (-3.1).

➡ Live demo: “ Q^* : the action scorecard”, revealed row by row

If you know Q^* , you know what to do

The table is worth the trouble because the optimal policy can be **read off** it:

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

If you know Q^* , you know what to do

The table is worth the trouble because the optimal policy can be **read off** it:

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

- In words: at state s , compare the values $Q^*(s, a)$ of all available actions and play an action with the largest value; $\arg \max$ denotes exactly that action.

If you know Q^* , you know what to do

The table is worth the trouble because the optimal policy can be **read off** it:

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

- In words: at state s , compare the values $Q^*(s, a)$ of all available actions and play an action with the largest value; $\arg \max$ denotes exactly that action.
- For the fleet: RUN when HEALTHY, SERVICE when WORN, REPLACE when SHAKY or FAILING.
- It remains to **compute the table**.

The Bellman equation

Suppose an optimal policy produced the trajectory $\dots, s_i, a_i, r_i, s_{i+1}, a_{i+1}, \dots$.
Averaging both sides of the recurrence $G_i = r_i + \gamma G_{i+1}$ over many such executions:

The Bellman equation

Suppose an optimal policy produced the trajectory $\dots, s_i, a_i, r_i, s_{i+1}, a_{i+1}, \dots$. Averaging both sides of the recurrence $G_i = r_i + \gamma G_{i+1}$ over many such executions:

The Bellman equation

$$Q^*(s_i, a_i) = \mathbb{E}[r_i + \gamma Q^*(s_{i+1}, a_{i+1})]$$

In words: the value of this week's state and action equals this week's reward plus γ times the value of next week's state and action, averaged over the randomness of the week.

Twelve equations, twelve unknowns

- One Bellman equation per state-action pair: twelve for the fleet.
- The right-hand side involves the same unknowns $Q^*(s, a)$ as the left-hand side: a system of equations in the entries of the table.

Twelve equations, twelve unknowns

- One Bellman equation per state-action pair: twelve for the fleet.
- The right-hand side involves the same unknowns $Q^*(s, a)$ as the left-hand side: a system of equations in the entries of the table.
- We can find Q^* by **solving the Bellman equations**. The visualization solves them iteratively, sweeping the table until the numbers settle.

Twelve equations, twelve unknowns

- One Bellman equation per state-action pair: twelve for the fleet.
- The right-hand side involves the same unknowns $Q^*(s, a)$ as the left-hand side: a system of equations in the entries of the table.
- We can find Q^* by **solving the Bellman equations**. The visualization solves them iteratively, sweeping the table until the numbers settle.
- Notable: the *decisions* settle long before the *values* do. The greedy playbook is already optimal after two sweeps.

➡ Live demo: “The Bellman equation” and “Filling Q^* with DP”

Two walls between us and the scorecard

Solving the Bellman equations requires **knowing the odds**: averaging over the week's outcomes means knowing how likely each outcome is.

Two walls between us and the scorecard

Solving the Bellman equations requires **knowing the odds**: averaging over the week's outcomes means knowing how likely each outcome is.

Two walls

- **Nobody hands you the odds**: no manual lists the breakdown odds for *your* van on *your* routes.
- **Real tables explode**: odometer, age, weeks since service, route load, forty vans: millions of states, half a billion scorecard cells.

Two walls between us and the scorecard

Solving the Bellman equations requires **knowing the odds**: averaging over the week's outcomes means knowing how likely each outcome is.

Two walls

- **Nobody hands you the odds**: no manual lists the breakdown odds for *your* van on *your* routes.
- **Real tables explode**: odometer, age, weeks since service, route load, forty vans: millions of states, half a billion scorecard cells.

What the manager *does* have is the **logbook**: week after week of lived experience. The state the van was in, the call made, the money booked, the state it landed in, the call made there.

➡ Live demo: “Why DP doesn’t scale”

SARSA's idea: estimate, then nudge

- Maintain a table $q(s, a)$ of **estimates**: current best guesses of $Q^*(s, a)$.
- Twelve cells for the fleet, every cell starting at 0.

SARSA's idea: estimate, then nudge

- Maintain a table $q(s, a)$ of **estimates**: current best guesses of $Q^*(s, a)$.
- Twelve cells for the fleet, every cell starting at 0.
- After every lived week, **improve the single cell just used**.
- No odds anywhere: the logbook alone feeds the table.

Pass one: forget the future for a moment

Simplest version of the question: what does *one* week of a given pair pay on average, say (WORN, RUN)?

- The odds are unknown, but the logbook shows what those weeks actually paid (mostly profits, the occasional breakdown):

+72, +72, +72, -208, +72, ...

Pass one: forget the future for a moment

Simplest version of the question: what does *one* week of a given pair pay on average, say (WORN, RUN)?

- The odds are unknown, but the logbook shows what those weeks actually paid (mostly profits, the occasional breakdown):

+72, +72, +72, -208, +72, ...

- Averaging them is the manager's reflex. **No filing cabinet** needed: keep one number per cell and nudge it toward each new week:

$$q(s_i, a_i) \leftarrow q(s_i, a_i) + \alpha[r_i - q(s_i, a_i)]$$

- The arrow is an instruction (replace the left side by the right side). α is the **learning rate** (0.15 in the visualization). Never overwrite, only nudge.

You already run this formula

It is exponential smoothing

The standard forecast-revision update from demand planning and spreadsheet forecasting:

$$\text{new forecast} = \text{old forecast} + \alpha \times (\text{actual} - \text{old forecast})$$

Revise the plan by a fraction of the miss, and never bet everything on the latest data point.

You already run this formula

It is exponential smoothing

The standard forecast-revision update from demand planning and spreadsheet forecasting:

$$\text{new forecast} = \text{old forecast} + \alpha \times (\text{actual} - \text{old forecast})$$

Revise the plan by a fraction of the miss, and never bet everything on the latest data point.

- SARSA forecasts the **value of a decision** the way a planner forecasts sales.
- If α shrinks like $1/n$, the nudges reproduce *exactly* the average of the weeks seen so far; held at a small constant such as 0.15, they produce an average that gently favors recent weeks.

Pass two: the “actual” grows up

But $Q^*(s_i, a_i)$ is not the average of one week's cash alone. By the recurrence $G_i = r_i + \gamma G_{i+1}$, a week is its cash *plus* γ times the value accumulated from the pair where the van lands.

Pass two: the “actual” grows up

But $Q^*(s_i, a_i)$ is not the average of one week's cash alone. By the recurrence $G_i = r_i + \gamma G_{i+1}$, a week is its cash *plus* γ times the value accumulated from the pair where the van lands.

- The future has not happened yet, and Q^* is exactly what we do not know. So the table's **current best guess** $q(s_{i+1}, a_{i+1})$ stands in for it: the same move as plugging a forecasted cash flow into an NPV calculation.

Pass two: the “actual” grows up

But $Q^*(s_i, a_i)$ is not the average of one week's cash alone. By the recurrence $G_i = r_i + \gamma G_{i+1}$, a week is its cash *plus* γ times the value accumulated from the pair where the van lands.

- The future has not happened yet, and Q^* is exactly what we do not know. So the table's **current best guess** $q(s_{i+1}, a_{i+1})$ stands in for it: the same move as plugging a forecasted cash flow into an NPV calculation.

The SARSA update

$$q(s_i, a_i) \leftarrow q(s_i, a_i) + \alpha [r_i + \gamma q(s_{i+1}, a_{i+1}) - q(s_i, a_i)]$$

In words: the same forecast revision, only the “actual” grew up: this week's money plus the discounted value of where the week left you.

Watch one nudge land

The bracket is the **surprise** (the planner's "miss"): the gap between what this week's evidence says the pair is worth and what the table currently believes.

Watch one nudge land

The bracket is the **surprise** (the planner's "miss"): the gap between what this week's evidence says the pair is worth and what the table currently believes.

Week one, table still all zeros

Logbook line: (HEALTHY, RUN, +95, WORN, SERVICE).

■ evidence: $95 + 0.9 \cdot q(\text{WORN}, \text{SERVICE}) = 95 + 0.9 \cdot 0 = 95$

■ surprise: $95 - 0 = 95$

■ the cell moves: $q(\text{HEALTHY}, \text{RUN}): 0 \rightarrow 0 + 0.15 \cdot 95 = 14.25$

One cell moves; the other eleven stay put.

➡ Live demo: "SARSA: learn from the logbook" at slow speed, one update per week

Why this solves the Bellman equation

$$\text{Bellman: } Q^*(s_i, a_i) = \mathbb{E}[r_i + \gamma Q^*(s_{i+1}, a_{i+1})]$$

Why this solves the Bellman equation

$$\text{Bellman: } Q^*(s_i, a_i) = \mathbb{E}[r_i + \gamma Q^*(s_{i+1}, a_{i+1})]$$

$$\text{SARSA: } q(s_i, a_i) \leftarrow q(s_i, a_i) + \alpha[r_i + \gamma q(s_{i+1}, a_{i+1}) - q(s_i, a_i)]$$

Why this solves the Bellman equation

$$\text{Bellman: } Q^*(s_i, a_i) = \mathbb{E}[r_i + \gamma Q^*(s_{i+1}, a_{i+1})]$$

$$\text{SARSA: } q(s_i, a_i) \leftarrow q(s_i, a_i) + \alpha[r_i + \gamma q(s_{i+1}, a_{i+1}) - q(s_i, a_i)]$$

- The Bellman equation says Q^* is the **average** of the evidence $r_i + \gamma Q^*(s_{i+1}, a_{i+1})$ over many weeks. An average over many executions is exactly how the expectation was defined.
- SARSA computes that average the only way available without the odds: **incrementally, as the weeks arrive**. The expectation replaced by an arriving average, Q^* replaced by the current guess q .

Why this solves the Bellman equation

$$\text{Bellman: } Q^*(s_i, a_i) = \mathbb{E}[r_i + \gamma Q^*(s_{i+1}, a_{i+1})]$$

$$\text{SARSA: } q(s_i, a_i) \leftarrow q(s_i, a_i) + \alpha[r_i + \gamma q(s_{i+1}, a_{i+1}) - q(s_i, a_i)]$$

- The Bellman equation says Q^* is the **average** of the evidence $r_i + \gamma Q^*(s_{i+1}, a_{i+1})$ over many weeks. An average over many executions is exactly how the expectation was defined.
- SARSA computes that average the only way available without the odds: **incrementally, as the weeks arrive**. The expectation replaced by an arriving average, Q^* replaced by the current guess q .
- The name spells the five logbook entries each update consumes: **S**tate, **A**ction, **R**eward, **S**tate, **A**ction.

Explore a little

Which calls should be made while the table is still being learned?

- Mostly the one the table currently favors: at state s , play the action with the largest $q(s, a)$.

Explore a little

Which calls should be made while the table is still being learned?

- Mostly the one the table currently favors: at state s , play the action with the largest $q(s, a)$.
- But in a small random share of the weeks (a fraction $\varepsilon = 0.15$ in the visualization) the call is drawn at random.

Explore a little

Which calls should be made while the table is still being learned?

- Mostly the one the table currently favors: at state s , play the action with the largest $q(s, a)$.
- But in a small random share of the weeks (a fraction $\varepsilon = 0.15$ in the visualization) the call is drawn at random.

Why experiment at all?

A cell that is never visited is never improved: you cannot learn the value of servicing if you never service.

Explore a little

Which calls should be made while the table is still being learned?

- Mostly the one the table currently favors: at state s , play the action with the largest $q(s, a)$.
- But in a small random share of the weeks (a fraction $\varepsilon = 0.15$ in the visualization) the call is drawn at random.

Why experiment at all?

A cell that is never visited is never improved: you cannot learn the value of servicing if you never service.

- The a_{i+1} in the update is the action *actually* chosen at s_{i+1} by this very playbook. SARSA learns the value of what it really does, occasional experiments included.

Does it work?

Run this, week after week, on one endless stream of lived weeks: **drive, get billed, nudge one cell.**

- The table drifts toward Q^* ; reading off the best action in each row recovers the playbook.
- In the visualization the same three policy bands emerge within a few hundred weeks of raw experience, **the odds never shown.**

Does it work?

Run this, week after week, on one endless stream of lived weeks: **drive, get billed, nudge one cell.**

- The table drifts toward Q^* ; reading off the best action in each row recovers the playbook.
- In the visualization the same three policy bands emerge within a few hundred weeks of raw experience, **the odds never shown.**

One honest caveat

With α and ε held constant, the twelve numbers stay rough: the cells settle below Q^* and keep jittering. But which call tops each row (all the playbook reads) comes out right.

➡ Live demo: “SARSA: learn from the logbook” at turbo speed, the bands appear

The whole journey

- **MDP** (S, A, T, γ) : states, actions, randomized dynamics, discounting
- **Policy** $\pi: S \rightarrow A$: a standing instruction for every state
- **Trajectory**: the random record of executing the policy, week by week
- **Return** $G_1 = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$: the discounted score of one trajectory
- **Objective**: maximize the average score $\mathbb{E}[G_1]$
- **$Q^*(s, a)$** : the scorecard, the expected return of acting optimally after starting with a in s
- **Bellman**: $Q^*(s_i, a_i) = \mathbb{E}[r_i + \gamma Q^*(s_{i+1}, a_{i+1})]$, twelve equations that pin down the table
- **SARSA**: when the odds are unknown, learn the same scorecard from the logbook alone, exponential smoothing applied to the Bellman equation

Play with it

The fastest way to make all of this concrete:

`sml-fs26.github.io/classic_rl/repair-or-replace`

- run the fleet for a quarter with hidden odds
- drag the γ slider and watch patience change the playbook
- watch the Q^* scorecard fill in, row by row
- let SARSA rediscover the three bands from nothing but driving, billing, and breakdowns

Play with it

The fastest way to make all of this concrete:

`sml-fs26.github.io/classic_rl/repair-or-replace`

- run the fleet for a quarter with hidden odds
- drag the γ slider and watch patience change the playbook
- watch the Q^* scorecard fill in, row by row
- let SARSA rediscover the three bands from nothing but driving, billing, and breakdowns

Same twelve numbers, same three bands, scene by scene, in the same order as this lecture.